

Phase 2-Report

What is Hardhat

Hardhat is a powerful development framework that automates the process of compiling, deploying, testing, and debugging smart contracts. It allows developers to work with local blockchain simulations (via *Hardhat Network*) and deploy to live testnets or mainnets when ready.

Key Features

- **Local Blockchain Simulation:** Enables testing and debugging without real gas fees.
- **Plugin Ecosystem:** Integrates tools like Ethers.js, Mocha, and Chai for testing.
- **Error Traceability:** Provides detailed stack traces for contract failures.
- **Network Flexibility:** Supports multiple networks (Hardhat, Localhost, Sepolia, etc.).

Simple Smart Contract in Hardhat:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleStorage {
    string private message = "Hello";

    function setMessage(string memory _message) public {
        message = _message;
    }

    function getMessage() public view returns (string memory) {
        return message;
    }
}
```

This contract allows a user to store and retrieve a message.

- `setMessage()` updates the message (transaction → costs gas).
- `getMessage()` reads the message (view function → free).

Simple UI :

Multi Network Smart Contract Execution

Select Mode:

Simulation Mode (Hardhat Local Node) ▼

Connect Wallet ⓘ

Enter new message

Set Message ⓘ

Get Message ⓘ

Multi Network Smart Contract Execution

Select Mode:

Testnet Mode (MetaMask)



Connect Wallet ⓘ

Enter new message

Set Message ⓘ

Get Message ⓘ

User Interface (UI) Overview and Functionality

1. Overview

The DApp features a **single-page interface** that allows users to deploy and interact with the *SimpleStorage* smart contract.

It is designed to be intuitive for both beginners (via Simulation Mode) and advanced users (via Testnet Mode).

The UI consists of:

- A **mode selector** to switch between Simulation and Testnet modes.
- A **wallet connection button** to link MetaMask or the local Hardhat node.

- A **message input field** to enter text for the setMessage function.
- **Set Message** and **Get Message** buttons to interact with the contract.

All interactive buttons remain disabled until a **valid contract connection** is established.

2. Modes and Connection Flow

2.1 Simulation Mode (Hardhat Local Node)

- Connects automatically to the **local Hardhat blockchain**.
- Deploys or loads the *SimpleStorage* contract when the app starts.
- Once connected, the **Set** and **Get** buttons are enabled for interaction.
- Provides a fast, gas-free testing environment for beginners.

2.2 Testnet Mode (MetaMask Connection)

- On selecting Testnet Mode, the DApp connects via **MetaMask**.
- It first tries to load an existing **contract address** from browser localStorage.
- If no saved address exists, it looks for a **default contract address** in frontend/config.json.
- If both are missing or invalid (e.g., the address belongs to a wallet/EOA), it **auto-deploys** a new *SimpleStorage* contract, saves the address, and connects to it.
- The **Set/Get** buttons are enabled once a valid connection is confirmed.

3. Inputs and Actions

Message Input

- Accepts a string value that will be sent to the contract's setMessage() function.

Set Message

- Sends a transaction to update the stored message on the blockchain.
- Disabled until a contract is connected.
- On success, the notification “ Message updated” is displayed.

Get Message

- Calls the getMessage() function to retrieve the currently stored message.
- Disabled until a contract connection is confirmed.

4. Status Indicators and Feedback

Wallet Status

- Displays “Connected” along with the connected wallet address after a successful connection.

Contract Status

- Displays “Contract deployed at ...” followed by the active contract address and mode (Local/Testnet).

Notifications and Alerts

Type	Example Messages
Success	“Connected to saved/default testnet contract”, “Contract deployed”, “Message updated.”
Validation	“That address is a wallet (not a contract)” when a non-contract address is detected.
Error	Clear messages for insufficient funds, network mismatch, or RPC issues.

5. Behavior and Logic Details

Address Management

- Once a contract is deployed on the testnet, its address is stored in **localStorage**.
- On subsequent visits, the app auto-connects using the saved address.
- A **default address** can be specified in frontend/config.json for instant connection.

Button States

- All buttons remain **disabled** until a valid contract connection is detected.
- Buttons are **enabled** immediately after a contract address is confirmed.

Consistency Across Modes

- The **UI layout and logic remain consistent** between Simulation and Testnet modes.
- Both flows use the same interaction components (input box, buttons, notifications).

6. Demonstration Procedure

Simulation Mode

1. Run a local blockchain:
2. `npx hardhat node`
3. Open the DApp and select **Simulation Mode**.
4. Connect wallet (local Hardhat account).
5. Enter "Hello" in the message box and click **Set Message**.
6. Click **Get Message** to confirm the update.

Testnet Mode

1. Switch to **Testnet Mode** in the UI.
2. Connect MetaMask (ensure test tokens are available).
3. Observe:
 - Auto-connect to a saved/default contract, or
 - A single new deploy prompt (if no contract found).
4. Interact using **Set Message** and **Get Message**.
5. Confirm: Contract address and status messages appear correctly.

7. Summary of UI Design Principles

Design Element	Purpose
Mode Selector	Allows users to switch between Hardhat simulation and live testnets.
Auto-deploy Logic	Simplifies usage for beginners by removing manual setup steps.
Visual Feedback	Provides transparency through clear messages and alerts.
Consistency	Ensures identical user experience in both modes.
Simplicity	Keeps UI minimal and educational for first-time blockchain learners.

Conclusion:

The implemented UI effectively bridges backend blockchain functionality with user-friendly interaction. By supporting both local simulation and live testnet modes, it accommodates learners, developers, and testers alike, ensuring a clear understanding of smart contract deployment and interaction workflows.